

INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

SESIÓN 3 Segmentación

I. DATOS GENERALES DE LA ASIGNATURA

Asignatura:	Diseño de Aplicaciones para Dispositivos Móviles
Período Académico:	IV Semestre
Créditos:	3 créditos
Duración en Horas:	40 horas
Docente:	Amaury Giovanni Méndez Aguirre

CONTEXTO: AISLAR EL OBJETO DEL FONDO

En la clase anterior analizamos el histograma de una imagen. Ese análisis estadístico tiene un propósito mayor: la **Segmentación**. Antes de que una Inteligencia Artificial pueda clasificar una célula cancerígena o reconocer la matrícula de un vehículo, el sistema debe ser capaz de separar matemáticamente el "Objeto de Interés" (Foreground) de su "Fondo" (Background).

El método más rápido y computacionalmente económico para lograr esto es la binarización (convertir la imagen a blanco y negro absolutos). Sin embargo, en el mundo real, la iluminación rara vez es perfecta, lo que genera "ruido" en nuestra segmentación. Para limpiar este ruido sin usar costosos algoritmos de Machine Learning, recurrimos a una rama de la geometría y el álgebra llamada **Morfología Matemática**.

1. Umbralización (Thresholding)

La umbralización es el proceso de binarizar una matriz en escala de grises. Se define un valor umbral T (Threshold). Evaluamos cada píxel $I(x,y)$ mediante una función escalón (función definida a trozos):

$$f(x, y) = 255, \text{ si } I(x, y) \geq T$$
$$f(x, y) = 0, \text{ si } I(x, y) < T$$

El desafío algorítmico es determinar el valor óptimo de T . Si elegimos un T muy alto, borraremos partes del objeto; si es muy bajo, incluiremos partes del fondo. Algoritmos avanzados como el **Método de Otsu** calculan estadísticamente este umbral minimizando la varianza intra-clase del histograma.

```
import cv2
import numpy as np

# Cargamos imagen directamente en escala de grises
imagen = cv2.imread('documento.jpg', cv2.IMREAD_GRAYSCALE)

# Binarización Estática (El programador define T = 127)
# Retorna dos valores: el umbral usado y la matriz resultante
_, imagen_binaria = cv2.threshold(imagen, 127, 255, cv2.THRESH_BINARY)

# Binarización de Otsu (La IA calcula el mejor T automáticamente)
T_otsu, imagen_otsu = cv2.threshold(imagen, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

print(f"El umbral óptimo calculado por Otsu fue: {T_otsu}")
```

TALLER ANALÍTICO 1: FUNCIÓN ESCALÓN (15 MIN)

Dada la siguiente sub-matriz I de 3×3 (intensidades de gris):

80	120	140
90	200	210
50	130	250

1. Aplique analíticamente una umbralización binaria con $T = 135$. Escriba la matriz resultante (solo debe contener 0 y 255).
2. Si el objetivo era aislar todos los valores mayores a 100, ¿cuál fue el error de elegir $T=135$? Explique qué le ocurriría visualmente al objeto en la imagen segmentada.

Espacio para la matriz resultante y justificación...

2. Operaciones Morfológicas: Erosión y Dilatación

Al binarizar, suelen quedar píxeles blancos aislados en el fondo (ruido de sal) o huecos negros dentro del objeto (ruido de pimienta). La Morfología Matemática corrige esto usando un **Elemento Estructurante (Kernel)**, que es una pequeña matriz booleana (usualmente de 3x3 o 5x5) que se desliza sobre la imagen.

- **Erosión:** El píxel central será blanco (255) *SOLO SI* todos los píxeles bajo el Kernel son blancos. Esto "adelgaza" las formas y elimina el ruido blanco aislado.
- **Dilatación:** El píxel central será blanco (255) *SI AL MENOS UNO* de los píxeles bajo el Kernel es blanco. Esto "engorda" las formas y rellena los huecos negros.

Elemento Estructurante (Cruz 3x3)

0	1	0
1	1	1
0	1	0

```
import cv2
import numpy as np

# 1. Definir el Elemento Estructurante (Kernel) usando NumPy
# Creamos una matriz de 5x5 llena de unos (Rectángulo)
kernel = np.ones((5, 5), np.uint8)

# Asumimos que 'imagen_binaria' ya tiene ruido
# 2. Aplicar Erosión (Elimina ruido externo)
# iteraciones = 1 significa que el kernel pasa una sola vez
imagen_erosionada = cv2.erode(imagen_binaria, kernel, iterations=1)

# 3. Aplicar Dilatación (Rellena huecos internos)
imagen_dilatada = cv2.dilate(imagen_erosionada, kernel, iterations=1)

# Nota: La combinación secuencial de Erosión seguida de Dilatación
# se conoce matemáticamente como operación de "Apertura" (Opening).
```

TALLER DE LABORATORIO FINAL: LIMPIANDO LA VISIÓN (45 MIN)

Desafío Práctico: Tratamiento de Ruido

1. Abran su IDE y carguen una imagen en escala de grises que tenga poco contraste (ej. un texto fotografiado con mala iluminación o una célula en microscopio).
2. Utilicen `cv2.threshold` con un valor estático que introduzca un poco de "ruido" intencional (puntos blancos no deseados en el fondo). Visualicen el resultado con `cv2.imshow`.
3. Construyan un Elemento Estructurante de 3x3.
4. Programen una operación morfológica de **Apertura (Erosión + Dilatación)** para limpiar el fondo sin encoger permanentemente el objeto principal.
5. Implementen una operación de **Cierre (Dilatación + Erosión)** en una variable diferente.
6. Muestren las 3 ventanas (Original binarizada, Apertura, Cierre) y concluyan: ¿Cuál operación morfológica fue más efectiva para su imagen específica y por qué?